• Technical Whitepaper & Architectural Blueprint

Apex LifeOS: Personal Finance & Operations ERP

Comprehensive Developer Specification, Security Governance Model, 13-Table Database Schema Reference, and Telemetry Engine Architecture.

RELEASE VERSION	FRONTEND STACK	BACKEND & DB	AI ENGINE
v2.4 High Performance	React 19 • Vite 8 • Tailwind v4	Supabase (PostgreSQL 15+)	Gemini 2.0 / 2.5 Flash

TABLE OF CONTENTS

1. Executive Overview & Personal ERP Philosophy
2. System Architecture & Tech Stack Topology
3. Security Architecture & Threat Defense
4. Comprehensive 13-Table Database Schema
5. Algorithmic Engines & Telemetry Logic
6. Edge Functions & Multi-Currency Engine
7. Android Companion Telemetry Subsystem
8. Deployment, CI/CD & Operations Runbook

01 Executive Overview & Personal ERP Philosophy

Traditional personal finance software operates asynchronously and reactively: users manually log expenses long after transactions occur, with zero awareness of spatial mobility, operational task commitments, or real-time context. This leads to high cognitive load, missed bill deadlines, delayed cash tracking, and incomplete records.

Apex LifeOS resolves this friction by operating as a **Personal Enterprise Resource Planning (ERP)** platform. It bridges three core domains into a unified, synchronized reactive runtime:

DOMAIN A: FINANCE

Liquid Ledger & Cash Vault

Multi-currency balance sheets, accounts, monthly budgets, physical cash/ATM wallet, and zero-shot AI transaction parsing.

DOMAIN B: MOBILITY

Telemetry & Geofencing

Continuous GPS velocity tracking, traffic-jam hysteresis filtering, fuel/fare estimation, and spatial geofence payment reminders.

DOMAIN C: OPERATIONS

Daily Task-Ledger Bridge

Payable bills, receivable milestones, recurring subscription burn rates, peer IOUs, and personal asset valuations.

02 System Architecture & Tech Stack Topology

Apex LifeOS is engineered with a modular, reactive architecture decoupled into distinct presentation, serverless compute, persistence, and telemetry tiers.



Frontend Architecture Highlights

- **React 19 & Vite 8:** Instant HMR, zero-bundle overhead with native ESM.
- **Tailwind CSS v4:** Dynamic theme engine supporting OLED Dark Mode and Light Mode.
- **Web Speech API:** 17+ spoken languages with interim transcript streaming and hands-free auto-commit.
- **Two-Row Navigation:** Complete decoupling of brand/account controls from module switching.

Backend & Security Highlights

- **Multi-Tenant Isolation:** Strict Row-Level Security (RLS) on all 13 tables.
- **Single-Device Lock:** Atomic session registry preventing concurrent session crosstalk.
- **Prompt Defense:** 4-tier sanitization preventing jailbreak and exfiltration vectors.
- **Zero-Hallucination Reset:** Clean starting balances at strictly 0.00.

03 Security Architecture & Threat Defense

Apex LifeOS handles sensitive personal telemetry, location trails, and financial records. The platform implements an end-to-end Zero-Trust posture across AI invocation, administrative governance, hardware sessions, and database access.

DEFENSE-IN-DEPTH

4-Tier AI Prompt Injection Mitigation Matrix

Natural language AI input fields are susceptible to adversarial prompt injection. Apex LifeOS executes a four-phase defensive barrier:

Tier 1: Pre-Execution Regex Scanner

Intercepts keywords including `ignore previous instructions`, `you are now DAN`, `system prompt`, `api_key`, and script tags before dispatching any network request to Gemini.

Tier 2: System Instruction Boundary Isolation

Prompts are never concatenated into a single string. The core persona and JSON extraction guidelines are loaded via Gemini's native `systemInstruction` parameter, while user text is enclosed in inert XML tags: `<untrusted_user_note>...</untrusted_user_note>`.

Tier 3: Post-Execution Strict Schema Validation

AI responses are structurally inspected. Numerical limits enforce transaction caps ($100,000,000.00), categories are mapped strictly to the approved 10-category taxonomy, and descriptions are stripped of HTML/script elements.

Tier 4: Client-Side Real-Time Prompt Guard

Real-time keystroke monitoring evaluates adversarial entropy. If risky sequences are detected, an alert badge is rendered and the submit button is programmatically disabled.

Role-Based Admin Governance (RBAC)

Platform governance and database wipe operations are hardcoded exclusively to the identity: `layanmoyura@proton.me`.

Public UI headers render no "Admin" buttons. Authorized entry is strictly granted via:

- Keyboard shortcut: `Ctrl + Shift + A`
- URL fragment router: `#admin`
- Settings modal conditional link (visible only to admin session)

Single-Device Session Enforcement

To prevent cross-device route contamination and conflicting balances, users are restricted to **one active hardware session** via table `user_device_sessions`.

- Logging into a new device registers the hardware fingerprint.
- Prior active devices are flagged and safely logged out.
- Background heartbeat sentinel continuously verifies ownership.

04 Comprehensive 13-Table Database Schema Reference

All tables are provisioned in PostgreSQL with native UUID primary keys, cascade referential integrity, and Row-Level Security (RLS) guaranteeing data tenancy.

#	TABLE NAME	PRIMARY KEY	FOREIGN KEYS & RELATIONS	RLS RULE SUMMARY
1	**accounts**	id (UUID)	user_id → auth.users	auth.uid() = user_id
2	**categories**	id (UUID)	user_id → auth.users	user_id OR is_default = TRUE
3	**transactions**	id (UUID)	account_id → accounts, category_id → categories	auth.uid() = user_id
4	**budgets**	id (UUID)	category_id → categories (UNIQUE with month_year)	auth.uid() = user_id
5	**tasks**	id (UUID)	account_id, category_id, related_transaction_id	auth.uid() = user_id
6	**personal_assets**	id (UUID)	linked_transaction_id → transactions	auth.uid() = user_id
7	**subscriptions**	id (UUID)	account_id → accounts, category_id → categories	auth.uid() = user_id
8	**contacts_ledger**	id (UUID)	settled_transaction_id → transactions	auth.uid() = user_id
9	**saved_places**	id (UUID)	category_id, linked_task_id, linked_contact_id	auth.uid() = user_id
10	**mobility_timeline_events**	id (UUID)	user_id → auth.users	auth.uid() = user_id
11	**daily_mobility_stats**	id (UUID)	user_id, stat_date (UNIQUE compound key)	auth.uid() = user_id
12	**device_location_pings**	id (UUID)	user_id, device_id (Hardware Telemetry Beacon)	auth.uid() = user_id
13	**user_device_sessions**	user_id (UUID)	active_device_id, last_active_at	auth.uid() = user_id

ZERO-HALLUCINATION RESET STORED PROCEDURE

```
-- PL/pgSQL Atomic Data Purge & Clean Account Initialization
CREATE OR REPLACE FUNCTION public.admin_purge_user_data(target_user_id UUID)
RETURNS JSONB LANGUAGE plpgsql SECURITY DEFINER AS $$
BEGIN
    DELETE FROM public.transactions WHERE user_id = target_user_id;
    DELETE FROM public.budgets WHERE user_id = target_user_id;
    DELETE FROM public.categories WHERE user_id = target_user_id AND is_default = FALSE
    DELETE FROM public.tasks WHERE user_id = target_user_id;
    DELETE FROM public.personal_assets WHERE user_id = target_user_id;
    DELETE FROM public.subscriptions WHERE user_id = target_user_id;
    DELETE FROM public.contacts_ledger WHERE user_id = target_user_id;
    DELETE FROM public.device_location_pings WHERE user_id = target_user_id;
    DELETE FROM public.saved_places WHERE user_id = target_user_id;
    DELETE FROM public.mobility_timeline_events WHERE user_id = target_user_id;
    DELETE FROM public.daily_mobility_stats WHERE user_id = target_user_id;
    DELETE FROM public.user_device_sessions WHERE user_id = target_user_id;
    DELETE FROM public.accounts WHERE user_id = target_user_id;

    -- Strict zero starting balance
    INSERT INTO public.accounts (user_id, name, balance, currency)
    VALUES (target_user_id, 'Primary Checking', 0.00, 'USD');

    RETURN jsonb_build_object('success', true, 'message', 'Purged & reset to 0.00');
END;
$$;
```

05 Algorithmic Engines & Telemetry Logic

The core intelligence of Apex LifeOS relies on deterministic spatial mathematics and stateful hysteresis filters.

1. Haversine Velocity Engine with Traffic Jam Hysteresis

Distance between consecutive GPS fixes (lat_1, lon_1) and (lat_2, lon_2) is computed via the great-circle Haversine formula:

```
d = 2R · arcsin( √( sin²(Δlat/2) + cos(lat1)·cos(lat2)·sin²(Δlon/2) ) )
velocity = (d / Δt) · 3600 [km/h]
```

The Traffic Jam Problem & Solution: Standard apps falsely reclassify car drivers waiting at red lights or crawling in bumper-to-bumper city traffic (0–15 km/h) as "pedestrians walking". Apex LifeOS maintains a motorized speed hysteresis buffer. If the vehicle previously maintained speed > 25 km/h within a sliding 10-minute window, deceleration below 15 km/h enters `Vehicle (Traffic Congestion)` state rather than `Walking`.

2. Geofence Sentinel & Proximity Triggers

Saved locations (Supermarkets, Landlord, Office) monitor spatial radii $r \in [100\text{m}, 1500\text{m}]$. When proximity threshold $d \le r$ evaluates to true:

- **Budget Category Sentinel:** Calculates remaining budget for that category and renders the global proximity alert banner.
- **Payable Task Trigger:** Generates a 1-click completion prompt to settle rent upon arrival at the landlord's office.
- **Social IOU Settle Trigger:** Prompts expense resolution when meeting a friend with an active balance.

3. The Financial Task Bridge

Tasks can be marked as `is_financial = true` with financial types: `payable` (bills), `receivable` (invoices), or `budget_task`. Checking off a task does not merely toggle a visual checkbox—it atomically logs a transaction against the designated account and updates current balances.

06 Edge Functions & Multi-Currency Engine

The natural language parser is deployed as a Deno TypeScript Edge Function on Supabase (`supabase/functions/parse-transaction/index.ts`).

PARSER API CONTRACT

```
// POST https://<project-ref>.supabase.co/functions/v1/parse-transaction
// Request Payload:
{
  "text": "Paid Rs 4,500 for weekly grocery run at Keells yesterday",
  "baseCurrency": "LKR",
  "categories": ["Groceries", "Dining Out", "Transportation", "Utilities", "Other
}

// Response JSON (HTTP 200):
{
  "amount": 4500,
  "currency": "LKR",
  "category": "Groceries",
  "type": "expense",
  "description": "Weekly grocery run at Keells",
  "date": "2026-10-18",
  "confidence": 0.98
}
```

Dynamic Currency Architecture

Apex LifeOS has zero hardcoded currency symbols. The application dynamically resolves formatting, currency symbols, and precision across 40+ global currencies (USD $, LKR Rs, EUR €, GBP £, INR ₹, JPY ¥, CAD CA$, etc.) using the browser's native `Intl.NumberFormat` engine synchronized through the user's system preferences.

07 Android Companion Telemetry Subsystem

The Android companion app (`android-companion/`) runs as a persistent native Kotlin background daemon, continuously streaming mobility metrics to Supabase without requiring an active browser tab.

Service Architecture

- **Foreground Service:** Ensures the OS does not terminate the telemetry service under memory pressure.
- **FusedLocationProviderClient:** High-precision GPS fixes with adaptive interval throttling.
- **Battery Throttling:** Dynamically backs off update frequency when the device is stationary.
- **Offline SQLite Queue:** Buffers pings locally if network connectivity drops.

Hardware Ping Payload Schema

```
{
  "device_id": "Pixel_9_Pro_a7b8c9",
  "device_name": "Pixel 9 Pro",
  "latitude": 6.927079,
  "longitude": 79.861244,
  "accuracy_meters": 4.2,
  "speed_kmh": 32.5,
  "battery_level": 82.0,
  "is_charging": false,
  "transport_mode": "vehicle"
}
```

08 Deployment, CI/CD & Operations Runbook

Commands and procedures for maintaining, testing, and deploying Apex LifeOS.

Local Development Setup

```
# 1. Clone repository and install dependencies
git clone https://github.com/layanpro123456/ai-finance-tracker.git
cd ai-finance-tracker
npm install

# 2. Configure .env file
VITE_SUPABASE_URL=https://<project-ref>.supabase.co
VITE_SUPABASE_ANON_KEY=<your-anon-public-key>

# 3. Start development server
npm run dev
```

Automated Quality Invariants & Build Verification

```
# Static analysis and linting (Oxlint)
npm run lint

# TypeScript compilation and production packaging
npm run build

# Generate Technical PDF Documentation
npm run docs:pdf
```

CI/CD Automation Pipelines (.github/workflows/)

- `deploy.yml`: Triggers on push to `main`, executes `npm run build`, and publishes production SPA bundle to GitHub Pages.
- `build-tracker-apk.yml`: Sets up Java 17 + Android SDK, builds debug and release APKs for the Kotlin telemetry companion.

Apex LifeOS Technical Blueprint • Architecture & Engineering
Authored for Engineering & Operations Teams